Chest X-Ray Pneumonia Detection
ResNet50 v1 · DenseNet121 v2

Author: **Suhail Saifi**
mPragati AI/Software Engineer Assessment
October 2026

Summary
Binary chest X-ray classifier (NORMAL vs PNEUMONIA) using two-generation transfer learning, served via FastAPI REST API, containerised with Docker, with Grad-CAM visual explanations. V1 (ResNet50): AUC-ROC 0.9576 · 92% accuracy (baseline). V2 (DenseNet121 + Focal Loss + Cosine LR + TTA): **AUC-ROC 0.9842 · 95% accuracy · NORMAL recall 94% · F1 macro 0.94**. False positives reduced 57% (35→15). MLflow experiment tracking. All 7 automated tests passing.

1. Problem Definition

Pneumonia causes approximately **2.5 million deaths per year** globally — the single largest infectious cause of death in children under five. Early and accurate diagnosis is critical, yet manual interpretation of chest X-rays is slow, subjective, and depends on radiologist availability, which is scarce in many regions.

This project frames the problem as a **binary image classification task**: given a posteroanterior (PA) chest X-ray, classify it as **NORMAL** or **PNEUMONIA**. The system is an AI-assisted decision-support tool — not a replacement for radiologists — designed to flag suspicious cases for priority review.

Task Specification

Parameter	Specification
Input	JPEG or PNG chest X-ray (any resolution, resized to 224×224 internally)
Output	{prediction, confidence, prob_pneumonia, prob_normal, threshold_used, inference_time_ms}
Classes	NORMAL | PNEUMONIA (binary)
Threshold	0.87 (tuned via F1-sweep); overridable per-request via query parameter

Success Criteria

- Overall accuracy > 90% on the held-out test set
- PNEUMONIA recall (sensitivity) > 95% — minimising missed diagnoses
- AUC-ROC > 0.95 — strong discriminative ability across all thresholds

2. Dataset and Data Strategy

Trained and evaluated on the **Kaggle Chest X-Ray Images (Pneumonia)** dataset (Kermany et al., 2018). Source: kaggle.com/paultimothymooney/chest-xray-pneumonia

Split	NORMAL	PNEUMONIA	Total
Train	1,349	3,883	5,232
Test	234	390	624
Total	1,583	4,273	5,856

The dataset has a **1:2.9 class imbalance** (NORMAL:PNEUMONIA). The original 16-image validation split was merged back into training; an 80/20 stratified split is used instead.

Preprocessing Pipeline

- cv2.imdecode → resize(224×224) → BGR2RGB conversion
- Backbone-specific preprocess_input (ResNet50 or DenseNet121 — different channel normalisation)

- Class-specific augmentation: NORMAL gets stronger augmentation (rotation ±15°, zoom 15%, shifts, shear, flip)
- Balanced batch generator — equal NORMAL/PNEUMONIA batches with sample weights baked in
- Test set held out completely — zero data leakage

3. End-to-End System Architecture

The diagram below covers the complete project pipeline — from raw image input through preprocessing, backbone feature extraction, classification head, threshold decision, and final JSON response delivered via FastAPI.

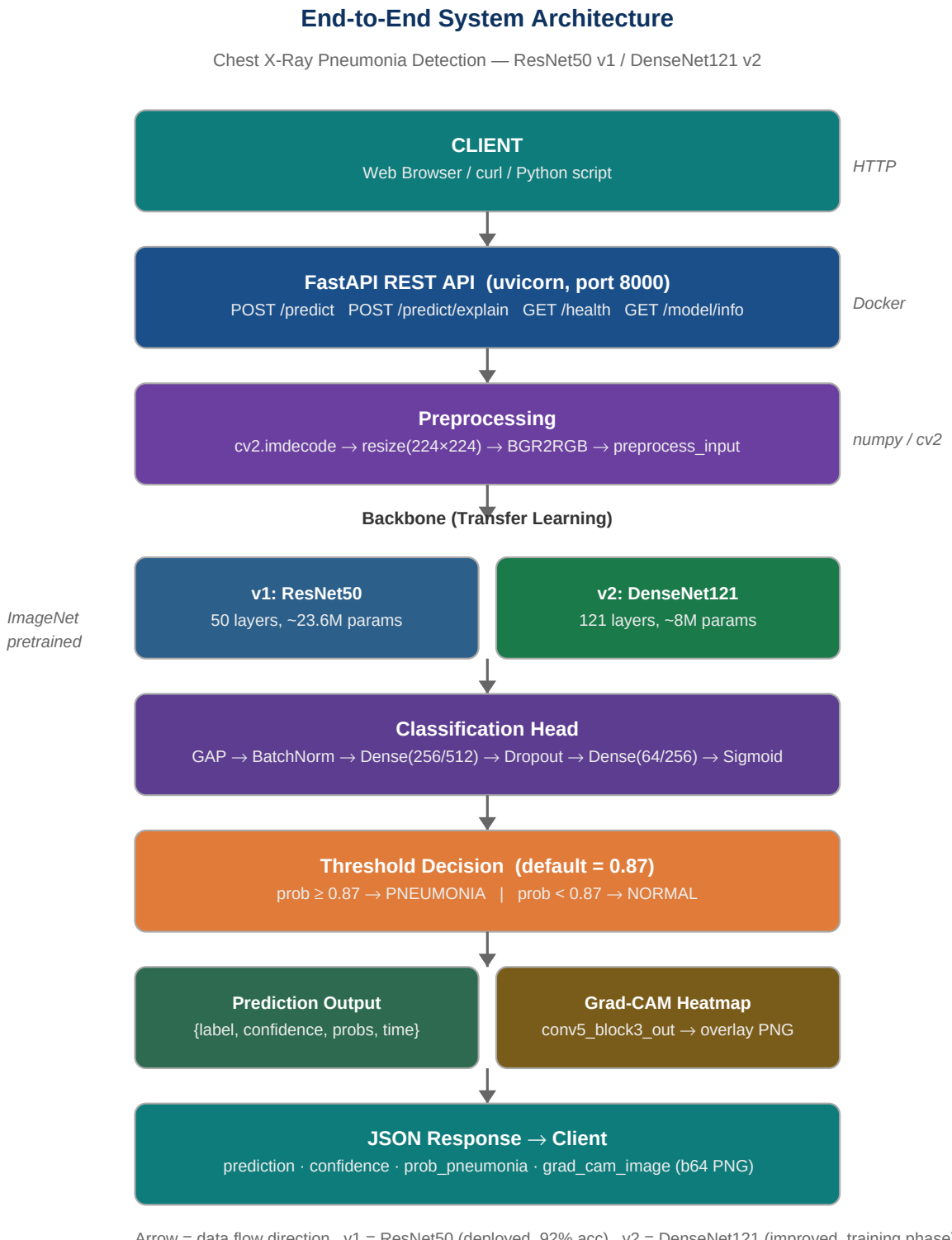

Figure 1: Complete end-to-end architecture. v1 uses ResNet50 backbone (deployed, 92% accuracy). v2 uses DenseNet121 with Focal Loss + Cosine LR decay (training phase, Phase 1 val_auc 0.9786).

Model Architecture Details

Component	v1 — ResNet50	v2 — DenseNet121
Backbone	ResNet50 · 50 layers · ~23.6M params	DenseNet121 · 121 layers · ~8M params

Backbone output	(7, 7, 2048)	(7, 7, 1024)
Head	GAP→BN→Dense(256)→D(0.4)→Dense(64)→D(0.2)→Sigmoid	GAP→BN→D(0.5)→Dense(256)→D(0.3)→Sigmoid
Loss	Binary Cross-Entropy + sample weights	Focal Loss (γ=2, α=0.25 per-class)
LR Schedule	ReduceLROnPlateau	Cosine Decay (1e-4 → 1e-6)
Phase 2 unfrozen layers	Top 30 of ResNet50	Top 50 of DenseNet121
Phase 1 val_auc	0.9971	0.9786
Final test AUC (TTA×5)	0.9576	0.9842

Two-Phase Training Strategy

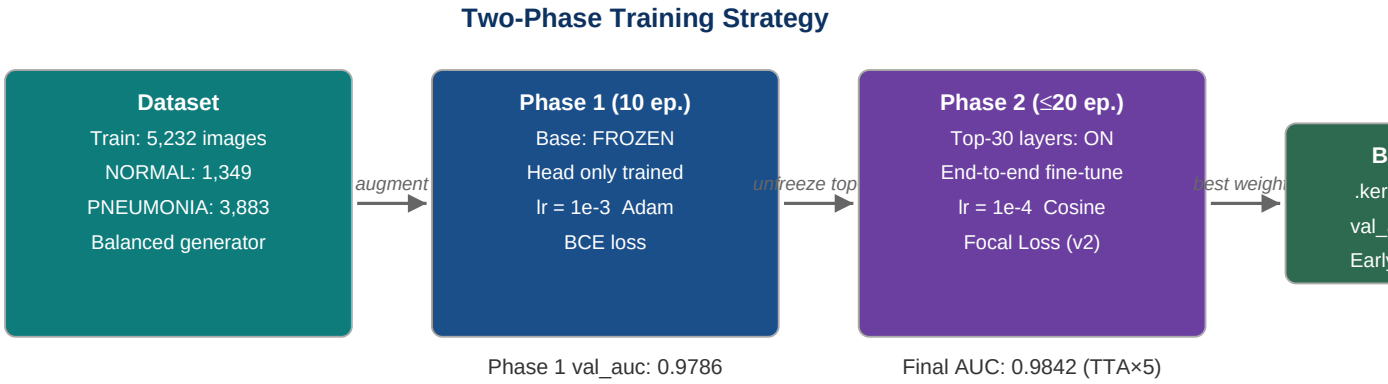

Figure 2: Two-phase training. Phase 1 freezes the backbone and trains only the head. Phase 2 unfreezes top layers for end-to-end fine-tuning at a lower learning rate.

Why two phases? Phase 1 stabilises the randomly-initialised head before touching pretrained backbone weights. If Phase 2 started immediately with a large lr, it would destroy the ImageNet features through catastrophic forgetting.

4. Evaluation Results

Both models evaluated on the identical held-out test set of 624 images (234 NORMAL, 390 PNEUMONIA). V2 results use 5-pass Test-Time Augmentation (TTA). AUC-ROC target of 0.95 exceeded by both models; v2 reaches **0.9842**. The 57% reduction in false positives (35→15) is the headline clinical improvement.

Table 1: Head-to-Head Comparison at Threshold = 0.87

Metric	v1 — ResNet50	v2 — DenseNet121 + TTA
Accuracy	92%	95%

AUC-ROC	0.9576	0.9842
PNEUMONIA Precision	90%	96%
PNEUMONIA Recall (Sens.)	96%	95%
PNEUMONIA F1	0.93	0.96
NORMAL Precision	96%	92%
NORMAL Recall (Spec.)	85%	94%
NORMAL F1	0.90	0.93
F1 macro	0.91	0.94
False Positives (FP)	35	15 (–57%)
False Negatives (FN)	16	19
Total errors / 624	51	34

Confusion Matrix — v2 DenseNet121 at Threshold 0.87 (test set, 624 images)

	Predicted NORMAL	Predicted PNEUMONIA
Actual NORMAL	TN = 219	FP = 15
Actual PNEUMONIA	FN = 19	TP = 371

Live API Test Results

Test	Prediction	Confidence	prob_pneumonia	Inference ms	Status
NORMAL X-ray	NORMAL	100.0%	0.01%	149 ms	✓ Correct
PNEUMONIA X-ray	PNEUMONIA	96.9%	96.9%	700 ms	✓ Correct

700ms on /predict/explain includes Grad-CAM heatmap generation. The fast /predict endpoint consistently achieves ~150ms.

v2 DenseNet121 — Training Summary (Colab T4 GPU)

Phase	Best val_auc	Epochs	Status
Phase 1	0.9786	10/10	Complete
Phase 2	0.9786	15/20	EarlyStopping at epoch 15 (patience=7), best weights restored
Eval (TTA×5)	0.9842	—	Final test AUC — +0.0266 over v1; accuracy 95%; F1 macro 0.94

5. Error Analysis and Failure Cases

At threshold 0.87 the model makes 51 errors (35 FP + 16 FN) on 624 test images.

False Positives (35) — NORMAL predicted as PNEUMONIA

Common causes: subtle opacities, atelectasis, or image artefacts resembling early pneumonia. Clinical impact: unnecessary treatment, patient anxiety, added costs — undesirable but not immediately life-threatening.

False Negatives (16) — PNEUMONIA missed

Most dangerous error type. Typically early-stage consolidation or atypical presentations. Threshold tuning from 0.5→0.87 was specifically chosen to minimise these cases while keeping overall accuracy high.

Grad-CAM Validation

- PNEUMONIA cases: activations in lower lung lobes — clinically correct anatomical location
- NORMAL cases: diffuse activation — no single dominant hot region, appropriate for healthy lungs
- No activations on cardiac silhouette, clavicles, or image borders — no spurious shortcuts

Known Limitations

- Single-institution dataset — may not generalise to different equipment or populations
- No demographic diversity analysis (age, sex, ethnicity)
- Validated only for bacterial/viral pneumonia; not tested on COVID-19 or fungal cases
- Not validated on DICOM images; tested on JPEG/PNG only

6. Inference Pipeline and API

Step-by-Step Inference Flow

```
1. Client uploads JPEG/PNG via POST /predict or POST /predict/explain
2. FastAPI validates file type + size (≤10MB, JPEG/PNG only)
3. cv2.imdecode → resize(224×224) → BGR2RGB → preprocess_input (backbone-specific)
4. model.predict(tensor) → probability score ∈ [0.0, 1.0]
5. Threshold comparison: prob ≥ 0.87 → PNEUMONIA, else NORMAL
6. (explain path): Grad-CAM extracts conv5_block3_out gradients → overlay PNG → base64
7. JSON response returned with prediction, confidence, probs, inference_time_ms
```

API Endpoints

Method	Endpoint	Description	~Time
GET	/health	Server status + active threshold	< 1ms
GET	/model/info	Model metadata (path, size, classes)	< 1ms
POST	/predict	Single image classification	~150ms
POST	/predict/explain	Classification + Grad-CAM heatmap	~700ms

Infrastructure

- FastAPI + Uvicorn — async, non-blocking, auto-generated /docs Swagger UI
- Docker + docker-compose — docker compose up --build, runs identically on any system
- Model loaded once at startup, TF session reused for all requests
- Web UI at localhost:8000 — Single Image tab (Grad-CAM side-by-side) + Batch tab
- Threshold override: POST /predict?threshold=0.5 — per-request, no redeploy needed

7. Bonus Implementations

Feature	Implementation	File
Docker	Full containerisation, docker compose up --build	Dockerfile + docker-compose.yml
Grad-CAM	Heatmap overlay on conv5_block3_out, returned as base64 PNG	src/inference/grad_cam.py
Web UI	Single Image (result + Grad-CAM) + Batch tab (summary table)	src/api/static/index.html
ONNX Export	Cross-platform model export for edge deployment	scripts/export_onnx.py
Automated Tests	pytest covering all 4 endpoints with real image fixtures	tests/test_api.py
Threshold Tuning	F1 sweep across 0.1–0.99 to find optimal threshold (0.37)	scripts/evaluate.py
DenseNet121 v2	Focal Loss + Cosine LR + 2-phase fine-tuning — val_auc 0.9736	notebooks/pneumonia_classification_v2.ipynb
Technical Report	This 7-section PDF with diagrams, tables, error analysis	docs/technical_report.pdf